SENG3030 – PROJECT PROTOTYPE

Team Members – Darsh Patel; Ayushkumar Rakholiya; Jal Shah; Viraj Solanki

INTRODUCTION

This project that we have made for our IOT course is about making a small alerting system that works over the internet using IoT ideas. We are using the devices that we have been using in this course to collect sensor data also to send it to an MQTT server and allow a control panel to manage what messages the devices listen to. The system uses sensors, a microphone, a display screen, and a speaker from the device which we are using. The main goal for this project is to see how devices can communicate to eachother by using the MQTT topics and react to the nmessages in real time.

The project that we have made shows us how the IoT systems gather the data and send it over to the network, react to commands, and show the results to the user. It also shows how an IoT device can be controlled remotely without touching the device directly. The overall project includes hardware, networking, MQTT topics, and simple user interaction.

SYSTEM OVERVIEW

The system we have made for this project has three main parts:

THE IOT DEVICES

These devices takes the sensor data every second and sends them to the MQTT server. By pressing the power button, they are able to switch data sending off and on. They are also able to record audio with the help of the microphone and then send that audio to the MQTT server where the front button is pressed.

THE MQTT SERVER

This server over here receives all the messages sent from the devices. Information is separated by using Topics and each message is stored in a topic name. These topics are also listened to by other devices and the control panel.

THE CONTROL PANEL

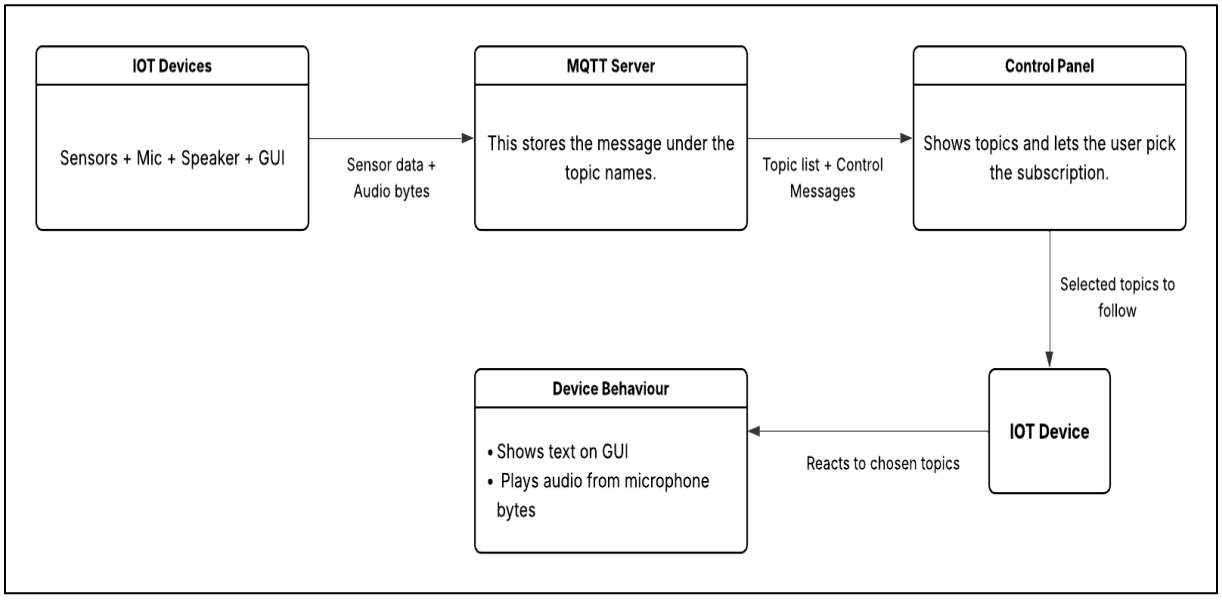
This is a program that allows us to choose which topics each device should subscribe to. It does not store any topics by itself. It listens to the MQTT server and finds topics from messages sent there. The user can select which ones the device should listen to.

PLANNED ARCHITECTURE

So heres the simple plan of how our project works like this:

1. First of the device strats by collecting the data from the sensors and sends it through the MQTT.
2. Then after the MQTT server further stores the messages under the topic names.
3. And the control panel watches these topics and displays them to the user.
4. Then the user further picks which topics the device should subscribe to.
5. The device receives new messages and reacts by:
   - Showing the text messages on screen.
   - Playing the audio if the message has microphone bytes in it.

Everything is connected through MQTT. No device talks directly to another device on themselves. The diagram below helps to get a clear idea of the planned architecture of the system we are designing for this project.



DEVICE GUI PLAN

The device screen is a very important part of this project. This has to be clean and easy to read. It should show the current message clearly without any confusion.

The GUI we have made will be behaving like this:

1. The screen will show that message whenever the device receives a new message from the MQTT topic right away.
2. The message will stay on the screen and will keep on updating till more data comes from the same topic.
3. The device will not be switching to other messages automatically untill and unless the user shakes it.
4. When the user will shake the device it means you want go to and see the next topic message on the display.
5. After shaking the device it will make a beep sound to show it is ready for the next message.
6. The colour in the background of the screen will also tell if the device is sending or not:
   - Green when sensors are sending data.
   - Blue when sending is off.

The idea over here is that the user does not have to press extra buttons or search menus. The device will react to what is coming from the MQTT server and shows the results in a simple way to the user.

CONTROL PANEL WORKFLOW

The control panel in this project is the main tool for managing the subscriptions of the topic. The device in starting has no subscriptions. The control panel gives the topics to the device.

Here is how the control panel works:

1. It will first connect to the MQTT server and then listen for all the messages under the course topic SENG3030.
2. Then after It collects the topic names from the server. These all show up on the control panel screen.
3. The user selects what topics the device should be subscribing to.
4. When the user picks a topic, the control panel sends a message to the device with the format of subscribe.
5. If the user wants to remove a topic, the control panel sends an unsubscribe message.
6. The device reads the subscribe/unsubscribe messages and resubscribes when a message is received.

This setup helps the device to change its behaviour without touching it physically. All decisions come from the control panel over MQTT.

MICROPHONE AND SPEAKER

The microphone and speaker helps in giving the audio feature for the project.

MICROPHONE

- The button in the front of the device controls the microphone action.
- Whenever the button is pressed the device sends the latest audio clip through MQTT.
- Before sending the clip to the MQTT the device first sends a smaller message telling the size of the audio.
- After that it sends the actual audio bytes.
- Because audio clips can be big sometimes the MQTT buffer has to be made bigger or the message will not fit into it.

SPEAKER

- When the device receives a message on a microphone byte topic, it plays the received audio on the speaker.
- This happens automatically.
- After shaking the device, a short beep plays to show the device is ready for the next message.

These steps helps in making the device able to share and listen to audio through the MQTT.

IMPLEMENTATION DETAILS

DATA COLLECTION IMPLEMENTATION

Our code first of all reads the temperature, humidity, pressure, battery level, and the IMU values from the device sensors. All of these value come from the SHT40, BMP280, and the built-in IMU. Then with the help of the read_unit_env() function over here will update the values every second. Then after, when the power button is clicked the device switches between sending and not sending. When sending is ON, the screen turns green and when sending is OFF the screen turns blue.

Every second when the sending is enabled the function publishSensorSnapshot() sends each value to its own MQTT topic. For example:

- SENG3030/Friday/<username>/sht40/temperature
- SENG3030/Friday/<username>/sht40/humidity
- SENG3030/Friday/<username>/battery

This means the values are always updated live on the server whenever the device is in the sending mode. The screen colour aslo helps the user understand if data is being sent or not.

MICROPHONE IMPLEMENTATION

The microphone over here keeps on recording in the background and stores the latest sound in a buffer. When the A button is clicked on the device the microphone stops recording and the saved audio is sent over to the MQTT server.

Because of the sound file is a bit large in size, it is futher broken into many smaller pieces called chunks. Each of the chunk over here is about 512 bytes, and all the chunks are then sent one by one on the /mic/bytes topic. So when all the chunks are sent then a message is sent on /mic/meta which informs that how many total bytes were sent and it will be in a format like, numbytes: 48000

This will tell all the other devices that how big the audio clip is going to be. So dividing the whole audio into smaller chunks solves the problem of the MQTT buffer being too small for one big message.

GUI IMPLEMENTATION

The GUI in this project helps in displaying the the latest message and the value on the screen everytime. It shows you the MQTT topic in small letters at the top and in the center is the value in large letters. The screen in this case changes only when there is a change in the topic or value. This helps in reduce flickering and this makes the display more stable.

Whenever there is a shake detected using the IMU then the device unlocks the topic and gets ready to show the next message that is comingup. After shaking the device makes a short beep sound to confirm it is ready. Until the device is shaken again, it will stay locked on the current message.

SPEAKER PLAYBACK IMPLEMENTATION

Whenever the device receives a message on: SENG3030/Friday/<username>/mic/bytes

It starts by playing the audio through the speaker. Then the device turns OFF the microphone temporarily so that there is no conflict. After the sound is done playing then the microphone turns back ON so it can record again in the background.

This means audio recorded on one device can be heard on another device without touching one of them physically.

CONTROL PANEL IMPLEMENTATION

The control panel implementation is basically a Python program that helps connect to the MQTT server and listens to all the messages under SENG3030/# topic. Whenever a message arrives the control panel reads the topic and sorts it by the device username. This lets the user see which topics each of the device is publishing.

The user can click on a topic in the list and that topic is placed into a text box. The control panel has two buttons: Subscribe and Unsubscribe.

When the user presses Subscribe, the control panel sends a message to: <username>/<device_user>/subscribe

with the topic filter the device should listen to. If the user presses Unsubscribe, the control panel sends a similar message to: <username>/<device_user>/unsubscribe

This allows the device to change its behavior without touching it. The control panel only shows messages from the topics the user has subscribed to, which makes it easy to monitor the exact data they want to see.

CHALLENGES AND SOLUTIONS

All of these helped make the system smooth and easy to use.

Challenge	**Why it happened**	**Our Solution**
Audio messages were too big	MQTT cannot send large messages in one go	We split the audio into 512-byte chunks
Screen flickering	Full redraws made GUI unstable	Only redraw when topic or value changes
Mic and speaker conflict	They cannot work at the same time	We turn the microphone OFF while playing sound
Too many topics	Hard to manage by hand	Control panel collects and lists topics automatically

REFERENCE

1. MQTT.org. (n.d.). *MQTT – The Standard for IoT Messaging*. https://mqtt.org/
2. OASIS Standard. (2019). *MQTT Version 5.0*. OASIS Open. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html
3. HiveMQ. (2023, June 6). *MQTT Essentials Part 2: Publish & Subscribe*. https://www.hivemq.com/blog/mqtt-essentials-part2-publish-subscribe/
4. EMQX. (2023, March 14). *Introduction to MQTT Publish-Subscribe Model*. https://www.emqx.com/en/blog/mqtt-5-introduction-to-publish-subscribe-model/
5. W3Schools. (n.d.). *Python tutorial*. W3Schools. https://www.w3schools.com/python/
6. Python Software Foundation. (n.d.). *Welcome to Python.org*. https://www.python.org/

EVALUATION FORM

Please include this page at the end of your document. All criteria are graded from zero to five.

Notes		Self Evaluation	Score
	Completeness, Format, References	5/ 5	5/ 5
Functionality		Self Evaluation	Score
	Data Collection	5/ 5	5/ 5
	Microphone	5/ 5	5/ 5
	Control Panel	5/ 5	5/ 5
	Device GUI	5/ 5	5/ 5
	Speaker Playback	5/ 5	5/ 5
Assignment Total			30/ 30